Machine Translated by Google

Guidelines for advanced internships / project work

Christoph Bockisch, Gabriele Taentzer (05.04.2018)

1. Procedure

For an advanced internship (Fopra) or project work (hereinafter both referred to as "project"), you should schedule at least four meetings with your supervisor at the beginning. You must also prepare a final report, which consists of the components listed below. You should discuss with your supervisor at the outset which of these components are relevant to your project (additional documentation requirements may also apply).

You should complete a partial document for each contact meeting (except the first two).
Please provide this approximately one week before the appointment so that we can then provide feedback.
In addition to the suggested deadlines for the documents below, individual implementation milestones should be agreed upon within the project and presented to the client/supervisor. Particular attention should be paid to ensuring that *at least one* such milestone is reached within the first semester, i.e., no later than approximately week 13 of the project.

> **Note:** For the aforementioned projects with the goal of developing software, an agile approach is typically recommended. In this case, more frequent meetings with the client/project manager are necessary. Furthermore, in an agile approach, the sub-documents are not fully created in advance, but are continuously expanded and revised throughout the project. Therefore, if necessary, submit updated versions of previously submitted sub-documents (clearly indicating the changes). The final version must be submitted for approval.

You may and should divide the creation of documents and other artifacts (such as code, models, etc.) among your team. However, all team members should be familiar with the content and able to answer questions. Each team member should be primarily responsible for creating at least part of a document and a portion of the implementation.

Please also note the *code requirements defined below.*

Contact appointments	FoPra appointment	Project work deadline
Topic presentation	Week 1 (W1): 1. Lecture week	Week 1 (W1): 2. Lecture week
Preliminary meeting	W2	W2
1st milestone	W4	W6
2nd Milestone	W8	W13
3rd Milestone	W14	W26
4th Milestone	W19	W37
Acceptance including commented code	W20	W38

Machine Translated by Google

For each contact appointment, you should create an updated schedule that becomes increasingly precise each time.

> **Note:** You should also track your effort, i.e., *when and how much time* you spent on *each task* . Only by comparing the actual effort with the estimated effort can you learn to assess your efficiency and improve your planning. In your schedule, indicate which planning goals you have already achieved and for which you have adhered to your time plan.

Hold regular team meetings and take minutes for each meeting.
The minutes should include which members attended, those with excused absences, and those who were absent without excuse. Briefly record the objectives, topics discussed, results, and agreements. To conduct team meetings efficiently, it is helpful to define the agenda items in advance. You should also determine beforehand who will take the minutes and whether this role is assigned permanently for the duration of the project or rotates (if so, establish a rotation schedule). After each meeting, please send the minutes to your supervisor and make them available to all team members.

2. Documentation requirements

2.1. Structure

The following defines content for potential documentation modules within the project, organized by various subject areas. You should aim to cover all subject areas in your project documentation. However, not all modules are suitable for every project, depending, for example, on whether the project is more practically or theoretically oriented, has a greater research or development focus, etc.

> **Note:** For a project where software development is the primary focus and which follows a waterfall approach, the following documents should each be created by the specified milestone at the latest (where the numbers such as "A1" or "A2" refer to the content described in more detail below):

1. Planning (for milestone 1)
    - (A1) Short description
    - (B1) Development process
    - (B2) Team
    - (B3) Risk management
    - (B4) Schedule
2. Requirements analysis (for Milestone 1)
    - (C1) Definition of the target system
    - (C2) Functional requirements
    - (C3) Non-functional requirements

3rd draft (for milestone 2)
    - (D1) Rough design (architecture)
    - (D2) Technologies
    - (D3) Detailed design
4. Quality Assurance (for Milestone 3)
    - (E1) Test plan
    - (E2) Test protocol
5. Final report (for milestone 4)

Machine Translated by Google

• (A2) Summary • (E3)
Example applications
• (F1) User documentation
• (F2) Developer documentation
• (A3) Experience report

Note: In an agile software development project, it is usually not possible to design the architecture in advance. Therefore, *the rough design (architecture) (D1)* should be moved to the *5th final report .* Furthermore, it is particularly recommended to create documents 2–4 in parallel and to submit a version for each milestone that covers the iterations completed so far as well as the immediately following one.

2.2. Building blocks

project

(A1) Short description

- Content description of the project
- Framework conditions or
  possibly embedded in a larger project
  - o Client: University/Working Group/Company
  - o special requirements
  - o time frame
  - o Quality requirements, e.g. scaling
  - o etc.

(A2) Summary

- Summary of the results achieved
- Possible future projects

(A3) Experience report

- Decisions that turned out to be good / bad
- Tips for future developers

Proceed

(B1) Development process

- Description of the software development process
- Practices to be applied (e.g., pair programming, test first, incremental, use cases or user stories, etc.)
- Documents to be created
- Fixed or rotating role assignment
- Programming language, modeling languages and other formats used (unless this can only be determined in the draft)
- CASE tools used
  - o Integrated development environment
  - o Modeling tools
  - o Continuous Integration Servers
  - o Test framework
  - o Test coverage

Machine Translated by Google

o Quality assurance tools (e.g., collecting code metrics, checking code conventions) o
Project management
tools o Configuration management o
Possibly other

(B2) Team

• Introduction of team members •
Tasks of team members or roles
according to the process description (e.g., Project Owner, Requirements Architect,
Quality Assurance
Responsibilities for documents, technologies, tools, protocols
etc.

(B3) Risk management

• Potential risks, e.g., unknown
technology, incomplete specifications or
components from the client, availability of team members (exams, vacation, illness?), heterogeneous
team composition, etc.

• Weighting of the identified risks
What is the probability that the risk will occur? How severe would
the impact be? Which risks need to be
addressed as a priority?
• Strategies for each identified risk: o Risk
avoidance o Risk
minimization o
Emergency plan

(B4) Schedule

• Availability planning •
Milestones / Releases •
Required specifications (components, libraries, specifications) with deadlines •
Release planning •
Planning goals achieved (Which deadlines were met/not met?) • Detailed planning for
short-term goals

Requirements

(C1) Definition of the target system

• Existing hardware on which the prototype to be developed must run • Existing
software (operating system(s), library(ies), etc.) with which the
The developing prototype must interact

(C2) Functional requirements

• Overview, e.g., using a use case diagram •
Documentation of all functional requirements according to the chosen process
(Use cases, user stories, etc.)
• Priorities (at least a division into "necessary" / "optional")

Machine Translated by Google

(C3) Non-functional requirements

- Quality of the system/code (performance, scalability, reliability, Extensibility, etc.) o Goals for code metrics o Goals for code conventions
- Usability (target group, ergonomics, etc.) • Technical requirements (technologies, tools, software, etc. to be used) • Documentation requirements • All non-functional requirements must be as specific and quantifiable as possible (testable) be specified.

Prototype design

(D1) Rough (software) design (architecture)

- Division of the implementation into components • Required and provided interfaces of the components • Architectural design patterns/strategies used • Selection of software/technologies to be used • Conventions/coding style

(D2) Technologies

- Technologies and techniques used: o Programming language o Existing software components o Libraries (version, URL, dependencies) o Frameworks (version, URL, dependencies) o Application Container (version, URL, dependencies) o Etc.

(D3) Detailed (software) design

- Detailed design of the components in UML diagrams
  - o At least one class diagram per component (focusing on important classes such as persistent data structures and services) o For more complex processes or behavior, one activity diagram each or state diagram
- Design patterns • Dependencies between components and with external components • Data model
- Developer responsibilities

(D4) Proof idea

- Requirements • Type of proof (direct or by contradiction)

(D5) Algorithm design

- Algorithm patterns (e.g., greedy, backtracking) • Coarse procedures as pseudocode

(D6) Database design

- Semantic data model (Entity-Relationship Model)

Machine Translated by Google

- Logical data model or tables

    - o Database definition using SQL

Validation

(E1) Test plan

- Test methods (GUI tests, unit tests, system tests, etc.) and what they are used for
    - Goals

for test coverage o Code coverage o

Specification coverage o

Justification for the stated goals

being sufficient • Description of the test cases developed so far

For programmatic tests, a very short description or a

"Title"

o For manual tests (GUI tests, acceptance tests), a description of the procedure

and expected result

- Characterization of the test cases yet to be developed

(E2) Test protocol

- Results of the (manual and automated) tests: o List of executed tests with results o List of

unexecuted tests • Report on the achieved test coverage

(E3) Example applications

- A slightly larger example that shows how the new result (software, algorithm,

etc.) can be used.

Instructions

(F1) User documentation

- Required dependencies (libraries, operating system, etc.) • Installation instructions
- Manual

(F2) Developer documentation

- For each architectural component, an overview description of the

Implementation •

Extension scenarios • Code

conventions/architecture to be observed • Known bugs, missing

features

3. Code Requirements

- Use the commenting standard and best practices of the programming language being used.

    - o E.g. JavaDoc, http://www.oracle.com/technetwork/articles/java/index-137868.html)

Machine Translated by Google

For example, Java code
conventions, https://google.github.io/styleguide/javaguide.html

- Code documentation:
  - Write a comment for each module you create.
  - o Write at least one comment for each public
    Function/Method (trivial, self-explanatory functions and methods, such as getter/setter, may be excluded)
  - Document complex passages in the code using inline comments.
- Always use meaningful and descriptive names for components,
  Classes, fields, methods, variables, etc.